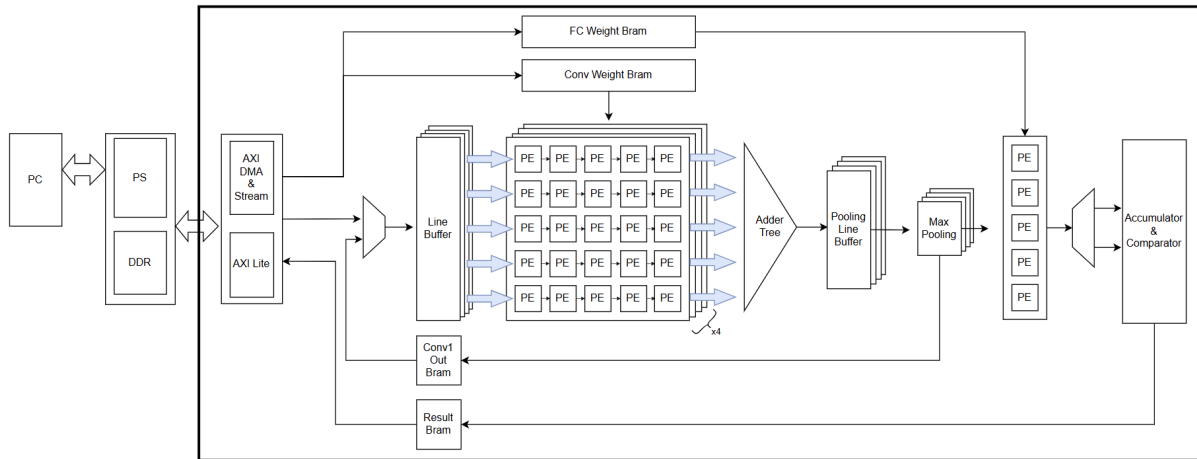


1. 전체 시스템 다이어그램

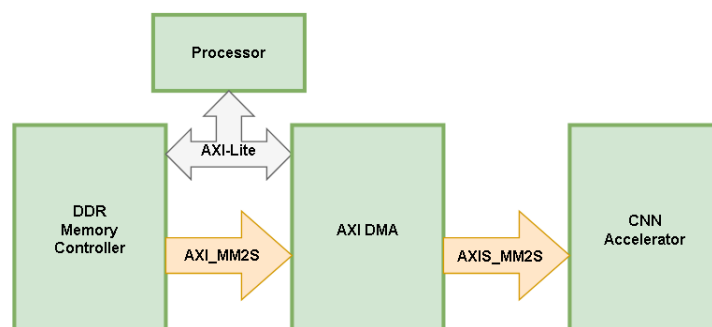


2. PS

AXI DMA

AXI DMA는 Zynq SoC에서 PS의 DDR에서의 대용량 데이터를 CPU의 개입 없이 PL로 전송할 수 있도록 해준다. Weight 데이터를 CNN 가속기 구동시 한번만 전송하며 conv1, conv2, fc layer의 weight가 하나의 연속된 버퍼로 구성된다. PS는 AXI DMA를 사용하여 해당 버퍼를 AXI-Stream 형태로 전송하며 이 때 DMA는 tdata[7:0]을 통해 8bit씩 총 3220번(conv1 100byte, conv2 1200byte, fc1 1920byte) 데이터를 전송한다. 가속기 내부 모듈은 입력 Stream을 수신하면서 5개씩의 8bit 데이터를 패킹하며 weight BRAM에 저장한다.

Weight 전송 완료 후 PS는 CNN 가속기의 start 신호를 통해 연산 상태로 전환하며 이후 AXI DMA는 입력 이미지 데이터 전송에 사용된다. PS는 AXI DMA를 통해 이미지를 8bit 단위로 784번 전송하며 총 1000회 반복으로 1000장의 이미지를 처리하도록 한다.



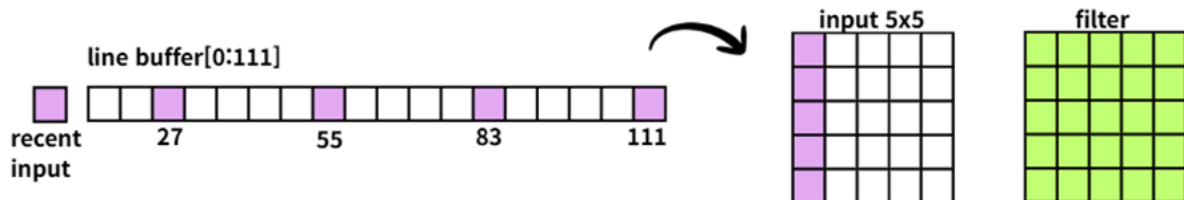
3. CONV 구성

weight_bram_reader

Weight_bram_reader 모듈은 conv1, conv2 연산에 사용되는 weight 1300개(conv1 100개, conv2 1200개)를 저장한다. BRAM에는 데이터가 8bit씩 5개를 1word로 하여 저장하고 있으며 파이프라인 방식으로 1clk마다 PE_array로 공급한다. Start가 1로 들어오면 IDLE에서 READ 상태로 전환되며, 내부 포인터가 가리키는 주소에 대해 bram_en과 bram_addr를 내보내 BRAM read를 요청한다. BRAM은 1clk 지연으로 데이터가 출력되기에 요청 다음 clk 부터 weight_valid를 1로 올려 해당 weight_out이 유효함을 표시한다. 이후 1word마다 데이터를 가져온다.

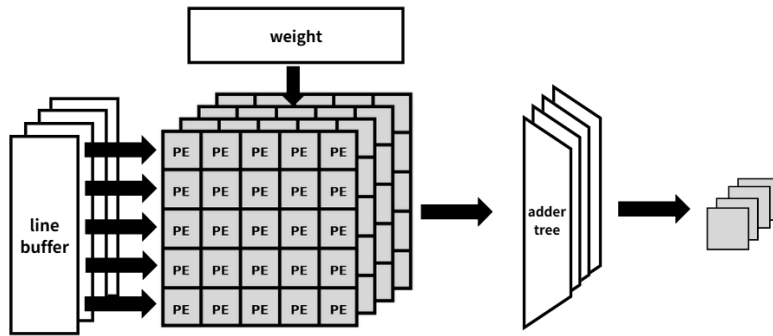
3-1 CONV

Line buffer



axi stream으로 입력 픽셀(8bit)가 1개씩 들어올 때마다 line buffer는 shift 하여 저장한다. Line buffer가 모두 채워진 순간부터 매 입력마다 현재 픽셀과 함께 1~4줄에 동일 열에 해당하는 픽셀을 각각 추출하여 세로 5개의 하나의 열을 생성한다. 이후 conv 모듈에 매 clk마다 총 5개의 열을 모두 보낸다. 이후 5x5 가중치 행렬과 함께 연산이 시작된다.

PE_array & adder_tree



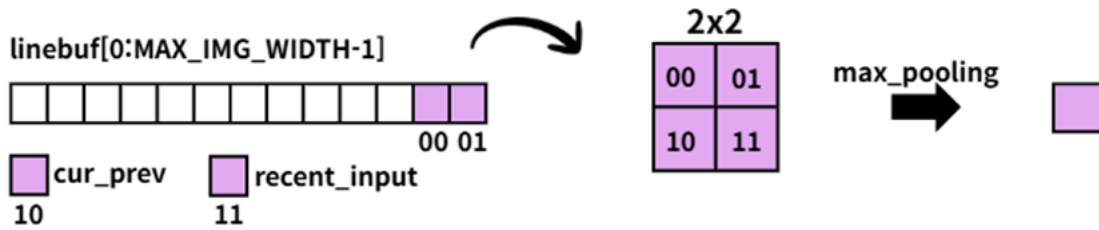
PE_array의 동작은 크게 weight 로딩 단계와 연산 단계로 나뉜다. 로딩 단계에서는 40bit로 들어오는 weight 한 묶음(8bit 5개)를 매 클럭마다 받아서 내부에 있는 PE에 한줄씩 채운다. 총 100개의 weight가 PE_array에 20번에 걸쳐 저장된다. 이후 연산 단계로 넘어가서 실제 입력을 받으며 계산한다. line buffer에서 입력값을 5개씩 PE_array로 전송한다. 5x5 PE array에 총 25개의 입력값이 각각 해당하는 weight 위치에 채워지고 각 입력 x 가중치 연산을 동시에 수행한다. 이후 곱 결과를 adder_tree에 전송한다.

adder_tree에서는 이전 PE_array를 통해 25개의 곱셈 결과를 전부 더한 뒤 결과 1개를 만든다. 총 5stage로 쪼개서 25개의 부분곱의 덧셈 연산을 수행한다. 이때 한번에 덧셈을 하지 않고 stage를 5단계로 나눈 이유는 한번에 16-bit 값 25개를 덧셈 연산 시, 긴 조합 논리로 인해 critical path가 길어지기 때문이다. 따라서 stage 별로 나눠 덧셈 연산을 진행했고, 총 4개의 adder_tree마다 1개 채널의 연산 결과를 출력한다.

3-2 requantization

Requantization 모듈은 conv 연산 결과를 다음 단계인 max pooling에서 사용 가능한 8bit unsigned 값으로 변환하기 위해 ReLU와 양자화 과정을 수행한다. 먼저 conv로부터 입력받은 22bit signed 값에 대해 ReLU를 적용하여 0 미만의 값을 제거하여 0으로 만들고, 0 이상의 값만을 다음 단계로 전달한다. 이후 양자화 과정에서는 round-to-nearest를 구현하기 위해 해당 값에 +64를 한 뒤 128을 나눈다. 마지막으로 8bit 범위 초과를 막기 위해 255를 넘으면 그대로 255를 out 값으로 하고, 이보다 작을 때는 하위 8bit 값을 out으로 한다.

3-3 pooling_line_buffer & max pooling



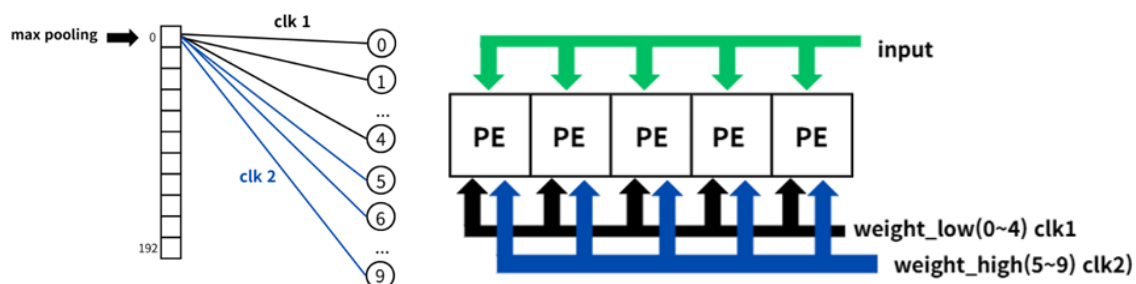
Pooling_line_buffer 모듈은 양자화를 거친 conv 출력 값을 1픽셀씩 입력 받는다. 이후 max pooling에 필요한 2x2 윈도우(out00~out11)를 스트리밍 방식으로 생성한다. 입력 폭은 conv1, conv2에 따라 24 또는 8이 될 수 있고, 이 사이즈에 맞춰 linebuf를 생성한다. Linebuf[col-1], linebuf[col]은 윗줄(00,01)을 의미하고, 그 다음 입력(10)은 cur_prev에 임시 저장한다. 이후 input(11)이 들어오면 앞서 00~10을 함께 결합해 2x2 output을 만든다.

Max_pooling 모듈은 pooling_line_buffer를 통해 받은 2x2값을 조합 논리로 서로 비교한 뒤 이 중 최대값을 1clk 뒤에 출력한다. 이때 총 4개의 채널이기에 4개의 max_pooling 모듈이 동시에 결과를 출력한다.

conv1_out_to_bram

conv1에서 max pooling을 마친 결과를 다시 bram에 저장한다. 저장된 값은 conv2에서 다시 읽어와 연산을 진행한다. conv2의 max pooling 결과는 따로 bram에 저장하지 않고 2clk마다 바로 FC layer로 보낸다.

3-4 FC layer

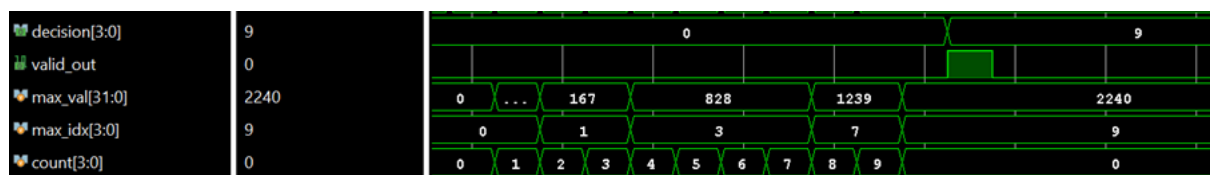


FC는 계층은 앞선 max pooling 계층을 거쳐 추출된 2차원 특징 벡터를 1차원 벡터 형태로 평탄화 한 뒤, 각 클래스에 대응되는 가중치와의 곱셈 및 누산 연산을 통해 최종 분류를 수행하는 계층이다. 본 설계에서는 max pooling 값을 2clk마다 하나씩 받아 이미지 분류를 수행하도록 구성했다. 하드웨어 자원 효율을 고려하여 Input 당 10개의 weight 중 5개씩

low(output 0~4), high(output 5~9) 나누어 처리하고 이에 2단계 파이프라이닝 된 PE 5개를 사용한다. 처음 input이 들어왔을 때 low weight 5개와 input이 5개의 PE의 입력으로 들어가고 곱 연산을 수행한다. 다음 clk에서 곱 결과가 각각의 acc[0] ~ acc[4]에 누산되며 동시에 PE의 weight입력은 high weight(5~9)으로 전환된다. 이후 곱 결과가 acc[5] ~ acc[9]에 누산된다. 모든 input에 대해 모든 가중치 곱이 완료되면 최종 output 값을 comparator로 1clk에 하나씩 보낸다. 모두 보내고 나면 FC_done 신호를 1로 만들어 연산 완료를 알리고 PE를 초기화 시킨다.

3 -5 comparator

비교기는 FC에서 계산된 10개의 출력값 중 최댓값을 갖는 클래스 index를 결정하는 모듈이다. FC로부터 1clk마다 output[0]부터 output[9]까지 순서대로 받는다. 이때 현재 값과 그 다음 들어오는 값을 1clk마다 비교하여 그 중 더 큰 값 만을 남긴다. 최종 비교가 완료되면, 가장 큰 출력 값의 index를 최종 분류 결과로 판단하고 이를 PS로 전달한다.



4. Weight Customization

4-1 Weight Learning Method

MNIST 로드 시에 학습용 transform에 대해 random의 회전, 이동시켜 보다 다양한 이미지에 대해 학습 가능하도록 한다.

```
train_transform = transforms.Compose([

    transforms.RandomAffine(degrees=5, translate=(0.05, 0.05)),

    transforms.PILToTensor()

])

test_transform = transforms.PILToTensor()
```

Weight중 절대값이 가장 큰 값을 기반으로 Symmetric Quantization 시킨다.

```
def fake_quant_weight_int8_ste(w_fp: torch.Tensor) -> torch.Tensor:
```

```
    alpha = w_fp.min()
```

```
    beta = w_fp.max()
```

```
    bound = torch.max(alpha.abs(), beta.abs())
```

```
    S = bound / 127.0
```

```
    S = torch.where(S > 0, S, torch.ones_like(S))
```

반올림으로 정수화된 결과를 다시 float로 복원(dequantize)하여 forward 연산에서 scale 비율에 따라 실제 int8 추론의 동작과 실질적으로 같은 연산이 되도록 하는 가중치로 사용하며(fake-quant) backward에서 weight 학습 시 원래의 float weight에 대해 gradient가 전달되도록 한다.

```
    q = torch.round(w_fp / S) # quantization
```

```
    q = torch.clamp(q, -128.0, 127.0)
```

```
    w_q = q * S # dequantization (int grid -> float)
```

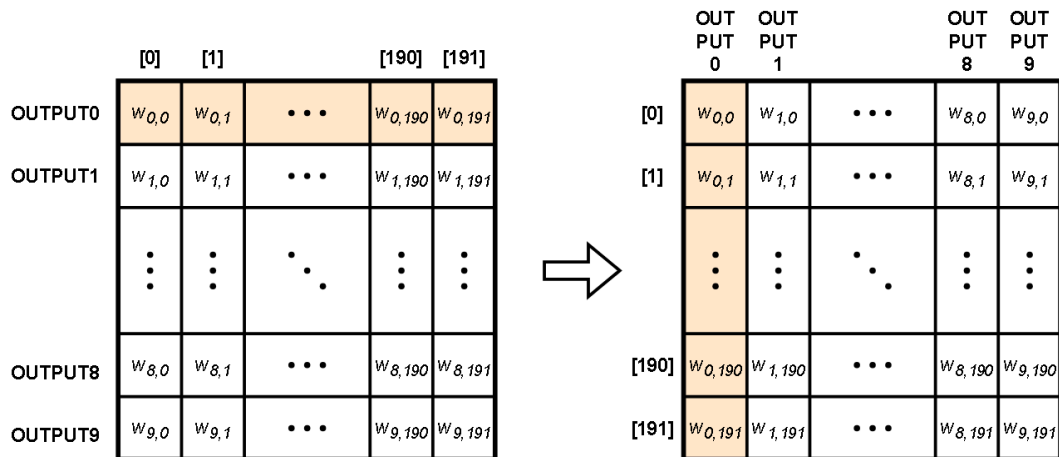
```
    return w_fp + (w_q - w_fp).detach() # use quantized value, train float weight
```

이러한 QAT 방식을 사용하여 최종적으로 int8 데이터로 requantize하여 header 파일을 export함으로써 학습에서의 높은 정확도를 유지할 수 있다. 이렇게 학습시킨 weight로 기존 test set 추론 정확도는 99.6%이다.

4-2 Weight Transpose

FC 연산에서 기존 제공된 FC1_arr.h의 weight 순서는 output 0을 출력하기 위한 값 192개가 순차적으로 입력된 후 output 1을 출력하기 위한 값 192개가 순차적으로 지정된 [10×192]의

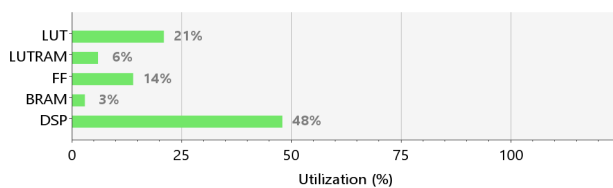
구성이다. 그러나 본 설계에서의 경우 **output 0~4** 및 **output 5~9**에 해당하는 **weight 5개씩**을 두 **phase**로 나누어 **PE**에 입력하며 곱 연산의 결과를 누적시키기에 모든 출력 **0~9**에 대한 **weight**가 연속적으로 제공되어야 하므로 **[192×10]**으로의 **Transpose**가 필요하다. 따라서 **ps**에서 연산하는 **weight**는 기존의 이름대로 **fc1_arr.h**, **pl**에서 연산하는 **weight**는 **fc1_arr_pl.h**로 선언하여 **weight** 순서만 바꾸어 연산이 되도록 하였다.



5. 결과

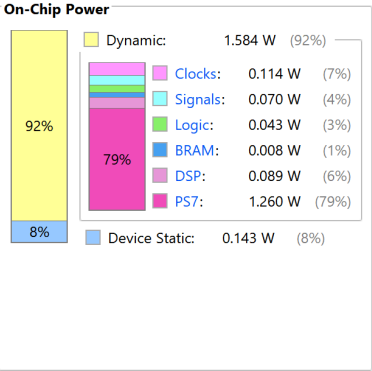
Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSP 1	Bonded IOPADs (130)	BUFGCTRL (32)
design_1_wrapper	10925	15310	194	4939	9808	1117	4	105	130	1
design_1_i (design_1)	10925	15310	194	4939	9808	1117	4	105	0	1
myip_v1_0_0 (design_1)	4479	8415	193	2574	4280	199	0	105	0	0
inst (design_1_myip_v1_0_0)	4479	8310	193	2556	4280	199	0	105	0	0
TOP_0 (design_1_wrapper)	4434	8222	193	2532	4235	199	0	105	0	0

Resource	Utilization	Available	Utilization %
LUT	10925	53200	20.54
LUTRAM	1117	17400	6.42
FF	15310	106400	14.39
BRAM	4	140	2.86
DSP	105	220	47.73



Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.727 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 44.9°C
Thermal Margin: 40.1°C (3.3 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 11.5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Medium
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.029 ns	Worst Hold Slack (WHS):	0.020 ns	Worst Pulse Width Slack (WPWS):	1.250 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	46782	Total Number of Endpoints:	46782	Total Number of Endpoints:	17136

All user specified timing constraints are met.

```
=== Summary ===
PS Acc = 996/1000 = 99.60% | Total = 8665609.56 us | Avg/img = 8665.61 us
PL Acc = 996/1000 = 99.60% | Weight upload = 39.58 us
PL Inference only: Total = 15482.21 us | Avg/img = 15.48 us
PL Cold-start total (upload + infer) = 15521.79 us
Speedup (PS/PL, steady-state avg) = 559.71x
```